

Practical Week-11

Task 1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
lenovo@Manva:~$ mkdir -p ~/OSSP/Threads
lenovo@Manva:~$ cd ~/OSSP/Threads
lenovo@Manya:~/OSSP/Threads$ pwd
/home/akshitha_reddy/OSSP/Threads
lenovo@Manya:~/OSSP/Threads$ nano race_condition.c
lenovo@Manya:~/OSSP/Threads$ gcc race_condition.c -o race_condition -pthread
lenovo@Manya:~/OSSP/Threads$ ./race_condition
Starting counter = 0
Expected counter = 4000000
Actual counter = 1478741
lenovo@Manya:~/OSSP/Threads$ ./race_condition
Starting counter = 0
Expected counter = 4000000
Actual counter = 1232946
lenovo@Manya:~/OSSP/Threads$ ./race_condition
Starting counter = 0
Expected counter = 4000000
Actual counter = 1061538
lenovo@Manya:~/OSSP/Threads$
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <sched.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;

        // Increase the chance of a race condition
        if (i % 1000 == 0)
            sched_yield();
    }
    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Starting counter = %d\n", counter);

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    // Wait for all threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter = %d\n",
           NUM_THREADS * ITERATIONS);

    printf("Actual counter = %d\n", counter);

    return 0;
}
```

Task2: Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
lenovo@Manva:~/OSSP/Threads$ nano mutex_counter.c
lenovo@Manva:~/OSSP/Threads$ gcc mutex_counter.c -o mutex_counter -pthread
lenovo@Manya:~/OSSP/Threads$ ./mutex_counter
Starting counter = 0
Expected counter = 40000000
Actual counter = 40000000
lenovo@Manya:~/OSSP/Threads$ |
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 10000000

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    // Initialize mutex
    pthread_mutex_init(&lock, NULL);

    printf("Starting counter = %d\n", counter);

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    // Wait for all threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter = %d\n",
           NUM_THREADS * ITERATIONS);

    printf("Actual counter = %d\n", counter);

    // Destroy mutex
    pthread_mutex_destroy(&lock);

    return 0;
}
```